

# Gerenciando dados textuais com pandas

Trabalhando com Texto em Python I — Unidade 5

Importar, examinar, estender e salvar DataFrames; aplicar PLN a colunas.

Adaptado de *Applied Language Technology* (Universidade de Helsinque) — [applied-language-technology.mooc.fi](https://applied-language-technology.mooc.fi)

## Objetivos de aprendizagem

Ao final desta unidade, você deverá saber:

- importar dados para um `DataFrame` do pandas;
- explorar dados armazenados em um `DataFrame`;
- acrescentar (estender) dados a um `DataFrame`;
- salvar os dados de um `DataFrame`.

O **pandas** é uma biblioteca do Python para trabalhar com **dados tabulares**. Começamos importando-a — usamos `as` para controlar o nome do módulo; por convenção, o pandas é abreviado como `pd`:

Python

```
import pandas as pd
```

## 1. Importando dados para o pandas

### 1.1 A partir de um único arquivo

Formatos típicos para distribuir corpora incluem **CSV** (*Comma-separated Values*), **JSON** e texto simples. O pandas oferece muitas funções de leitura — dá até para importar planilhas do Excel! O exemplo carrega um recorte do *SFU Opinion and Comments Corpus* (SOCC), com artigos de opinião do jornal canadense *The Globe and Mail*. A função `read_csv()` recebe o **caminho** do arquivo:

Python

```
# Le o arquivo CSV e atribui a saída a variável 'socc'
socc = pd.read_csv('data/socc_gnm_articles.csv')

# Examina o tipo do objeto guardado em 'socc'
type(socc)
```

#### Saída

```
pandas.core.frame.DataFrame
```

O pandas devolve o conteúdo do CSV em um `DataFrame`, sua estrutura de dados nativa. O método `head()` mostra as primeiras linhas:

#### Python

```
# Imprime as primeiras cinco linhas do DataFrame
socc.head(5)
```

O `DataFrame` tem forma **tabular**, com colunas (`article_id`, `title`, `author`, `article_text`...) e um **índice** por linha (0, 1, 2, ...). Versão compacta (algumas colunas omitidas para caber):

| índice | article_id | title                                             | author           | ntop_level_comments |
|--------|------------|---------------------------------------------------|------------------|---------------------|
| 0      | 26842506   | The Tories deserve another man-date...            | GLOBE EDITORIAL  | 1378.0              |
| 1      | 26055892   | Harper hysteria a sign of closed liberal minds    | Konrad Yakabuski | 455.0               |
| 2      | 6929035    | Too many first nations people live in a dream...  | Jeffrey Simpson  | 433.0               |
| 3      | 19047636   | The Globe's editorial board endorses Tim Hudak... | GLOBE EDITORIAL  | 432.0               |
| 4      | 11672346   | Disgruntled Arab states look to strip Canada...   | Campbell Clark   | 411.0               |

O acessador `.at[]` inspeciona **um único item**. O valor da coluna `title` no índice 123:

#### Python

```
socc.at[123, 'title']
```

#### Saída

```
"How Toronto got a 'world-class,' gold-plated, half-billion-dollar empty train"
```

## 1.2 A partir de múltiplos arquivos

Outro cenário comum é ter **vários arquivos** de texto. Primeiro, coletamos os arquivos com a classe `Path` (vista na Unidade 1):

#### Python

```
# Importa a classe Path
from pathlib import Path

# Cria um objeto Path que aponta para o diretório com os dados
corpus_dir = Path('data')

# Coleta todos os arquivos .txt no diretório do corpus
```

```
corpus_files = list(corpus_dir.glob('*.txt'))

# Verifica os arquivos do corpus
corpus_files
```

Saída

```
[PosixPath('data/WP_1990-08-10-25A.txt'),
 PosixPath('data/NYT_1991-01-16-A15.txt'),
 PosixPath('data/WP_1991-01-17-A1B.txt')]
```

Criamos um **DataFrame** **vazio** definindo a forma de antemão: o número de linhas (**index**, com **range()** de 0 até **len(corpus\_files)**) e os nomes das colunas (**columns**):

Python

```
# Cria um DataFrame e atribui o resultado a variavel 'df'
df = pd.DataFrame(index=range(0, len(corpus_files)), columns=['filename', 'text'])

# Chama a variavel para inspecionar a saida
df
```

| índice | filename | text |
|--------|----------|------|
| 0      | NaN      | NaN  |
| 1      | NaN      | NaN  |
| 2      | NaN      | NaN  |

Percorremos os **Paths**, lemos o conteúdo e o adicionamos com o acessador **.at**:

Python

```
# Percorre os arquivos do corpus e conta cada volta com enumerate()
for i, f in enumerate(corpus_files):

    # Le o conteúdo do arquivo
    text = f.read_text(encoding='utf-8')

    # Pega o nome do arquivo do objeto Path
    filename = f.name

    # Atribui o texto do arquivo ao índice 'i' na coluna 'text' usando
    # o acessador .at -- isso modifica o DataFrame "no lugar" (in place);
    # não é preciso atribuir o resultado a uma variavel
    df.at[i, 'text'] = text

    # Faz o mesmo com o nome do arquivo
    df.at[i, 'filename'] = filename
```

O **DataFrame** fica populado com os nomes dos arquivos e o texto:

| índice | filename               | text                                                |
|--------|------------------------|-----------------------------------------------------|
| 0      | WP_1990-08-10-25A.txt  | *We Don't Stand for Bullies': Diverse Voices...     |
| 1      | NYT_1991-01-16-A15.txt | U.S. TAKING STEPS TO CURB TERRORISM:<br>F.B.I. I... |
| 2      | WP_1991-01-17-A1B.txt  | U.S., Allies Launch Massive Air War Against T...    |

## 2. Examinando DataFrames

Os **DataFrames** guardam muita informação, organizada em **colunas**, acessíveis pelo atributo **columns**:

Python

```
# Recupera as colunas e seus nomes
socc.columns
```

Saída

```
Index(['article_id', 'title', 'article_url', 'author', 'published_date',
      'ncomments', 'ntop_level_comments', 'article_text'],
      dtype='object')
```

Uma coluna inteira é acessada com colchetes `[]` e o nome como *string* — como as chaves de um dicionário:

Python

```
# Recupera o conteúdo da coluna 'author' no DataFrame 'socc'
socc['author']
```

Saída

```
0      GLOBE EDITORIAL
1    Konrad Yakabuski
2    Jeffrey Simpson
...
10337  Adam Radwanski
10338  GLOBE EDITORIAL
Name: author, Length: 10339, dtype: object
```

Cada coluna é, na verdade, um objeto **Series** — pense no **DataFrame** como a tabela inteira, cujas colunas são **Series**:

Python

```
# Verifica o tipo de 'socc' e de 'socc['author']'
type(socc), type(socc['author'])
```

#### Saída

```
(pandas.core.frame.DataFrame, pandas.core.series.Series)
```

#### Nota

Ao imprimir um `DataFrame` ou `Series`, o pandas **omite** tudo entre as cinco primeiras e as cinco últimas linhas por padrão — conveniente com milhares de linhas.

O método `value_counts()` conta os valores únicos de uma `Series`:

#### Python

```
# Conta os valores unicos na coluna 'author'
socc['author'].value_counts()
```

#### Saída

```
GLOBE EDITORIAL          2712
Jeffrey Simpson          649
Margaret Wente           547
...
Jessica Scott-Reid       1
Kenneth Oppel            1
Name: author, Length: 1896, dtype: int64
```

Podemos **visualizar** o resultado com o método `.plot()`, que usa a biblioteca **matplotlib**. Passamos `bar` ao argumento `kind` e `[:10]` para os dez autores mais prolíficos:

#### Python

```
# Magia do Jupyter que permite renderizar graficos do matplotlib no notebook!
# Voce so precisa executar este comando uma vez.
%matplotlib inline

# Conta os valores da coluna 'author' e limita ao top-10 antes de plotar.
socc['author'].value_counts()[:10].plot(kind='bar')
```

#### Dica

O `.plot()` gera uma **imagem** (gráfico de barras) exibida no notebook; a saída de texto é apenas `<AxesSubplot: >`. Para outros tipos, mude `kind` (ex.: `'line'`, `'hist'`, `'pie'`).

Para colunas **numéricas**, o método `describe()` dá estatísticas descritivas:

#### Python

```
# Estatísticas descritivas basicas da coluna 'ntop_level_comments'
socc['ntop_level_comments'].describe()
```

#### Saída

```
count    10339.000000
mean      26.384273
std       39.786923
min        0.000000
25%        1.000000
50%       14.000000
75%       35.000000
max      1378.000000
Name: ntop_level_comments, dtype: float64
```

Lendo a saída: 10 339 linhas; média (**mean**)  $\approx 26$  comentários, com forte variação (desvio-padrão **std**  $\approx 40$ ); mínimo (**min**) 0; primeiro quartil (**25%**) = 1; mediana (**50%**) = 14; terceiro quartil (**75%**) = 35; máximo (**max**) = 1 378.

## 2.1 Filtrando linhas com `.loc`

Como achar os artigos com **zero** comentários? Usamos `.loc` para selecionar linhas por valor. Como `=` é reservado para atribuição, a comparação “é igual a” usa **dois** sinais (`==`):

#### Python

```
# Pega as linhas sem comentarios de nivel superior
socc.loc[socc['ntop_level_comments'] == 0]
```

#### Saída

```
[... 2542 rows x 8 columns ...]
```

Retorna 2542 linhas. Para visões mais complexas, combinamos critérios com `&` (“E” lógico) — cada critério entre **parênteses** (`()`):

#### Python

```
# Numero de comentarios de nivel superior para o autor Hayden King
socc.loc[(socc['ntop_level_comments'] == 0) & (socc['author'] == 'Hayden King')]
```

Retorna apenas a linha de índice 7797 — o único artigo dele sem comentários.

## 3. Estendendo DataFrames

É fácil **acrescentar** informação. Um cenário comum: carregar dados, analisar e guardar os resultados no mesmo `DataFrame`. Adicionamos uma coluna vazia com `[]` e o tipo `None`:

#### Python

```
# Adiciona uma nova coluna chamada 'comments_ratio' ao DataFrame
socc['comments_ratio'] = None
```

Preenchemos a coluna com a proporção de comentários de nível superior sobre o total — dividindo uma coluna pela outra:

Python

```
# Preenche a coluna 'comments_ratio' com a razão entre comentários de
# nível superior e o total de comentários
socc['comments_ratio'] = socc['ntop_level_comments'] / socc['ncomments']
```

Uma amostra do resultado:

| índice | ntop_level_comments | ncomments | comments_ratio |
|--------|---------------------|-----------|----------------|
| 0      | 1378.0              | 2187.0    | 0.630087       |
| 1      | 455.0               | 1103.0    | 0.412511       |
| 2      | 433.0               | 1164.0    | 0.371993       |

Mas alguns artigos **não** receberam comentários — nesses casos, dividiríamos zero por zero. Para essas linhas, `comments_ratio` fica marcada como NaN (*not a number*). O pandas **ignora automaticamente** os NaN nos cálculos, como mostra o `describe()`:

Python

```
# Estatísticas descritivas da coluna 'comments_ratio'
socc['comments_ratio'].describe()
```

Saída

```
count    7797.000000
mean      0.537057
std       0.205398
min       0.083333
25%      0.384615
50%      0.485714
75%      0.647059
max       1.000000
Name: comments_ratio, dtype: float64
```

Note o `count`: só 7 797 itens (dos 10 339) entraram no cálculo — os NaN foram ignorados.

### 3.1 Aplicando processamento de linguagem a uma coluna

E se quiséssemos fazer PLN e guardar os resultados? Seleccionamos artigos no primeiro quartil de `comments_ratio` e com mais de 200 comentários, usando `&`:

Python

```
# Filtra o DataFrame por artigos muito comentados e atribui o resultado a 'talk'
talk = socc.loc[(socc['comments_ratio'] <= 0.384) & (socc['ncomments'] >= 200)]
```

Importamos o spaCy e carregamos um modelo **médio** para o inglês:

Python

```
# Importa a biblioteca spaCy
import spacy

# Note que agora carregamos um modelo de tamanho medio!
nlp = spacy.load('en_core_web_md')
```

Ao criar uma coluna nova em `talk`, o pandas emite um `SettingWithCopyWarning`, porque `talk` é apenas um recorte (*slice*) do `DataFrame` original. A solução é criar uma **cópia profunda** com `.copy()`:

Python

```
# Cria uma copia profunda do DataFrame
talk = talk.copy()

# Cria uma nova coluna chamada 'processed_title'
talk['processed_title'] = None
```

Para processar os títulos, usamos `apply()`, que **aplica** o que recebe a cada linha da coluna. Passamos o modelo `nlp`:

Python

```
# Aplica o modelo de linguagem 'nlp' ao conteúdo da coluna 'title'
talk['processed_title'] = talk['title'].apply(nlp)
```

Cada célula da coluna `processed_title` agora contém um objeto `Doc` do `spaCy`:

Python

```
# Pega o valor da coluna 'processed_title' na linha de indice 2
talk.at[2, 'processed_title']

# Verifica o tipo do objeto contido
type(talk.at[2, 'processed_title'])
```

Saída

```
Too many first nations people live in a dream palace
spacy.tokens.doc.Doc
```

### 3.2 Definindo a própria função e aplicando com `apply()`

Vamos definir uma função para buscar os **lemas de cada substantivo** do título. Funções são definidas com `def`, seguido do nome e dos parâmetros entre parênteses:

Python

```
# Define uma funcao 'get_nouns' que recebe um unico objeto como entrada.
```



```
# Referimo-nos a essa entrada pela variavel 'nlp_text'.
def get_nouns(nlp_text):

    # Primeiro garantimos que a entrada e do tipo correto,
    # usando 'assert' para checar o tipo
    assert type(nlp_text) == spacy.tokens.doc.Doc

    # Prepara uma lista vazia para os lemas
    lemmas = []

    # Percorre o objeto Doc
    for token in nlp_text:

        # Se a classe gramatical detalhada do token for substantivo (NN)
        if token.tag_ == 'NN':

            # Acrescenta o lema do token a lista de lemas
            lemmas.append(token.lemma_)

    # Ao fim do laço, retorna a lista de lemas
    return lemmas
```

Aplicamos com `apply()`, criando a coluna `nouns` automaticamente pela atribuição:

```
Python

# Aplica a funcao 'get_nouns' a coluna 'processed_title'
talk['nouns'] = talk['processed_title'].apply(get_nouns)
```

Uma amostra dos títulos processados e seus substantivos:

| índice | title                                               | nouns            |
|--------|-----------------------------------------------------|------------------|
| 2      | Too many first nations people live in a dream. . .  | [dream, palace]  |
| 5      | Fifty years in Canada, and now I feel like a s. . . | [class, citizen] |
| 8      | A nation of \$100,000 firefighters                  | [nation]         |

#### Nota

Não é necessário criar uma coluna vazia antes — o pandas cria a coluna nova automaticamente na atribuição, como em `talk['nouns']`.

### 3.3 Extrair dados para estruturas nativas do Python

O método `tolist()` extrai o conteúdo de uma `Series` para uma **lista**:

```
Python

# Converte a pandas Series em uma lista
noun_list = talk['nouns'].tolist()

# Mostra os dez primeiros
noun_list[:10]
```

Saída

```
[[ 'dream', 'palace'], [ 'agency'], [ 'class', 'citizen'], [ 'right'],  
 [ 'nation'], [], [ 'reform'], [ 'leader', 'parade'], [ 'pm'],  
 [ 'government', 'monopoly']]
```

Temos uma **lista de listas**. Vamos “achatar” tudo em uma única lista `final_list` com `extend()`:

Python

```
# Prepara a lista vazia  
final_list = []  
  
# Percorre cada lista dentro da lista de listas  
for nlist in noun_list:  
  
    # Estende a lista final com a lista atual  
    final_list.extend(nlist)  
  
# Verifica o tamanho da lista  
len(final_list)
```

Saída

884

Para plotar os 10 substantivos mais frequentes, convertamos em `Series`, contamos com `value_counts()` e plotamos:

Python

```
# Converte a lista em uma pandas Series, conta os substantivos unicos com  
# value_counts(), pega os 10 mais frequentes [:10] e plota em barras.  
pd.Series(final_list).value_counts()[:10].plot(kind='bar')
```

## 4. Salvando DataFrames

`DataFrames` podem ser salvos como objetos **serializados** (*pickled*) com o método `to_pickle()`, que recebe o caminho do arquivo:

Python

```
# Grava o DataFrame em disco usando pickle  
df.to_pickle('data/pickled_df.pkl')
```

### Segurança (importante no CiberExt)

O formato *pickle* serializa objetos Python arbitrários e, ao ser **carregado**, pode **executar código arbitrário**. **Nunca** faça `read_pickle()` (nem `pickle.load`, `joblib.load`, `numpy` com

`allow_pickle=True` etc.) em arquivos de **fontes não confiáveis** — um `.pkl` malicioso compromete a máquina. Use *pickle* só com dados que **você mesmo** gerou; para intercâmbio entre pessoas/sistemas, prefira formatos que não executam código, como **CSV**, **JSON** ou **Parquet**.

Verificamos lendo de volta com `read_pickle()`:

Python

```
# Le o DataFrame serializado e atribui o resultado a 'df_2'
df_2 = pd.read_pickle('data/pickled_df.pkl')
```

Comparando os dois `DataFrames` com `==`, obtemos um booleano por célula:

Python

```
# Compara os DataFrames 'df' e 'df_2'
df == df_2
```

| índice | filename | text |
|--------|----------|------|
| 0      | True     | True |
| 1      | True     | True |
| 2      | True     | True |

Tudo `True` — os dados foram salvos e recarregados com sucesso.

## Quiz

Marque a alternativa correta (a certa está marcada com **(correta)**).

**1. Por convenção, o pandas é importado com qual apelido?**

1. `pd` **(correta)**
2. `pandas`
3. `dt`

**2. Cada coluna de um DataFrame é, na verdade, um objeto:**

1. `Series` **(correta)**
2. `DataFrame`
3. `list`

**3. Um valor NaN indica:**

1. Um valor ausente (“não é um número”) **(correta)**
2. O número zero
3. Infinito

**4. Qual operador combina dois critérios num filtro `.loc`?**

1. `&` **(correta)**

2. `and`

3. `+`

**5. Por que criar uma cópia com `.copy()` de um recorte do DataFrame?**

1. Para evitar o `SettingWithCopyWarning` e não alterar o original (**correta**)
2. Para acelerar o cálculo
3. É obrigatório em toda operação

**6. (Segurança) Por que *não* usar `read_pickle` em um arquivo de origem não confiável?**

1. Porque pode executar código arbitrário na sua máquina (**correta**)
2. Porque é mais lento que o CSV
3. Porque não consegue ler texto

## Resumo da unidade

### Nesta unidade, você aprendeu a

1. importar dados de um único arquivo (`read_csv()`) ou de vários (`Path/glob` + laço com `.at`);
2. examinar DataFrames: `columns`, colunas como `Series`, `value_counts()`, `describe()`, `.plot()` e filtragem com `.loc (&, ==)`;
3. estender: adicionar colunas, calcular entre colunas, lidar com `NaN`, usar `.copy()` e aplicar funções com `apply()`;
4. extrair com `tolist()/extend()` e salvar/carregar com `to_pickle()/read_pickle()` (com cuidado de segurança).

### Exercícios sugeridos

1. Carregue os três arquivos de notícias da Unidade 1 em um `DataFrame` (colunas `filename` e `text`) e conte quantos caracteres cada texto tem, numa nova coluna `n_chars`.
2. No `DataFrame socc`, use `.loc` para selecionar os artigos de `GLOBE EDITORIAL` com mais de 500 comentários.
3. Escreva uma função `get_verbs()` (análoga a `get_nouns`) que colete os lemas dos **verbos** (`token.pos_ == 'VERB'`) e aplique-a a uma coluna de títulos processados.
4. Salve um `DataFrame` em CSV com `to_csv('saida.csv', index=False)` e recarregue com `read_csv()`, conferindo as colunas.

**Referências.** *Applied Language Technology* (MOOC), Universidade de Helsinque — [applied-language-technology.mooc.fi](https://applied-language-technology.mooc.fi).  
Kolhatkar, V. et al. (2020). *The SFU Opinion and Comments Corpus*. Corpus Pragmatics. McKinney, W. et al. *pandas: powerful Python data analysis toolkit* — [pandas.pydata.org](https://pandas.pydata.org).

Material produzido para o Programa CiberExt 26-29 / FEELT38103 — Universidade Federal de Uberlândia.